hashlock.

2026

Penetration Testing

Arcane 2nd (DeFi)

July 2026

info@hashlock.com.au

Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY PENETRATION TEST REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

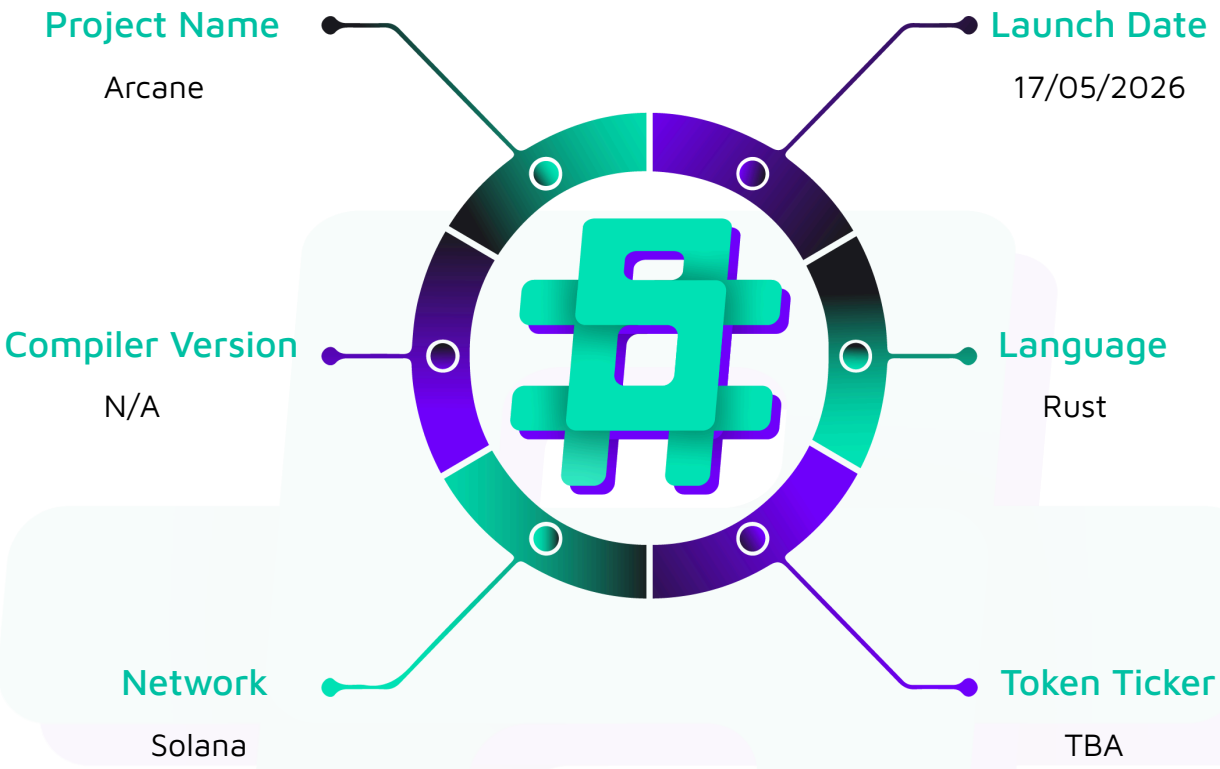
The Arcane team partnered with Hashlock to conduct a security Penetration Test of their Arcane Project code base. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed tools were secure.

Project Context

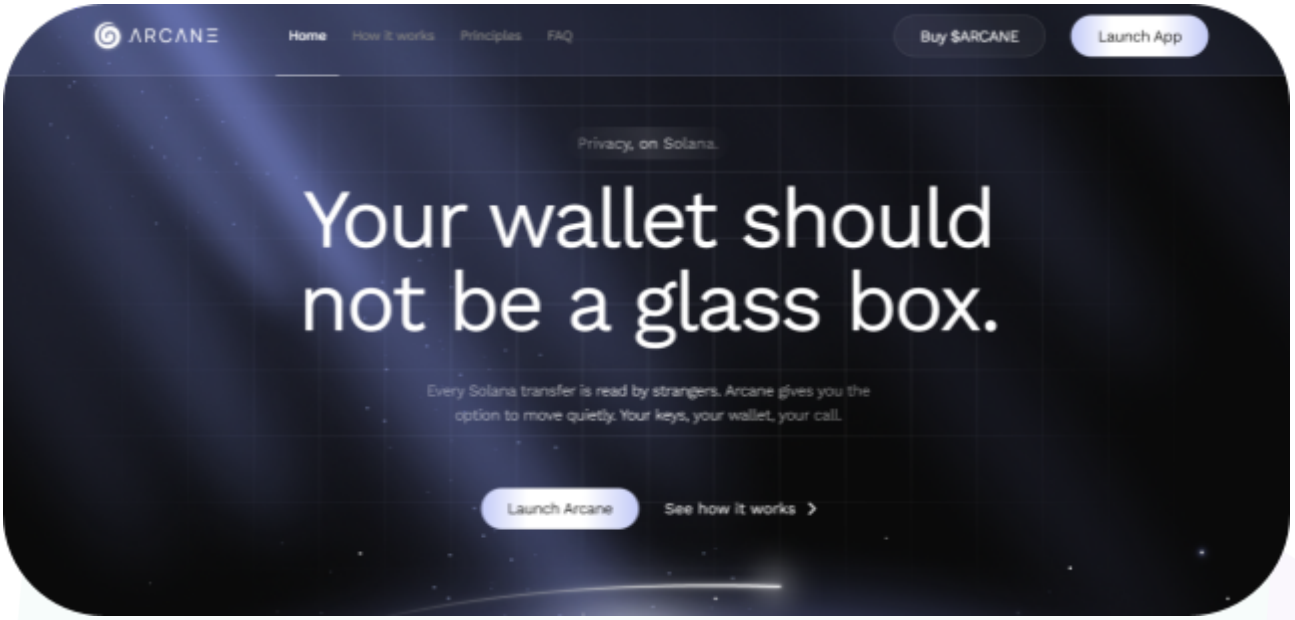
Arcane is a Web3 privacy-focused infrastructure project building confidential and compliance-oriented blockchain solutions for decentralized finance and cross-chain applications. The protocol focuses on enabling private transactions, shielded asset transfers, and privacy-preserving DeFi interactions through advanced cryptographic mechanisms and decentralized infrastructure. Arcane's ecosystem includes components such as private liquidity infrastructure, privacy-enhanced wallets, cross-chain interoperability mechanisms, and institutional-grade compliance tooling designed to support secure and confidential on-chain activity across multiple blockchain ecosystems.

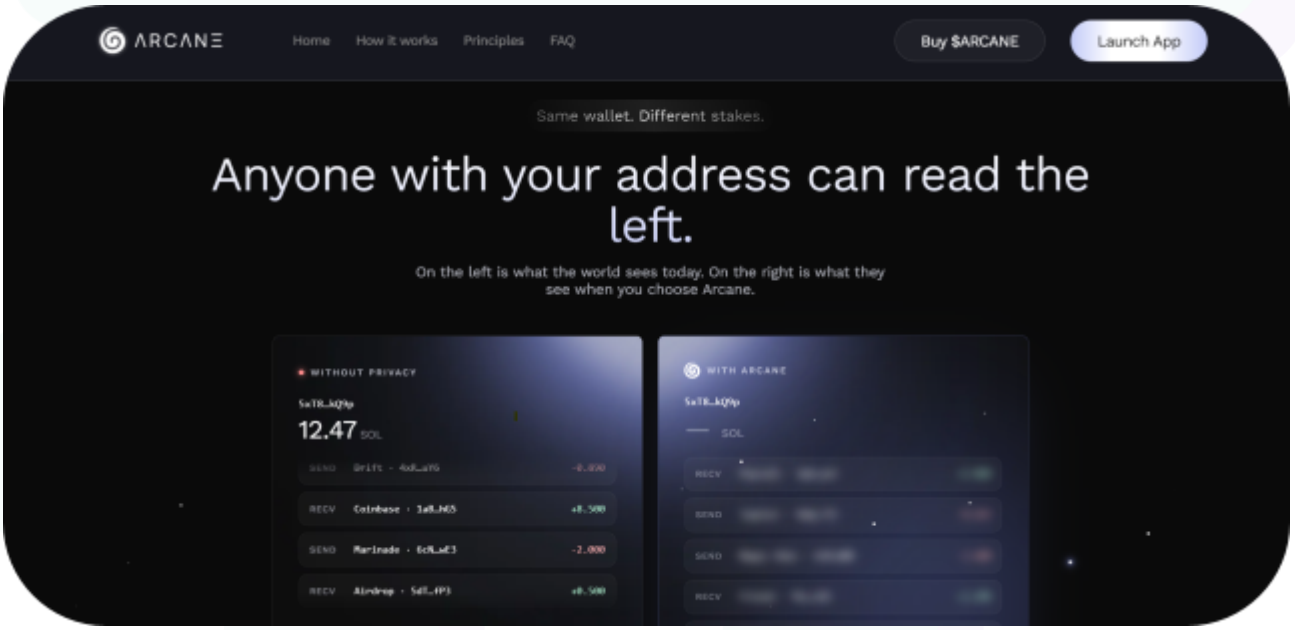
Project Name: Arcane
Project Type: DeFi
Website: https://arcaneprivacy.com/
Logo:

Visualised Context:



Project Visuals:





Penetration Test Scope

At Hashlock, we executed a comprehensive penetration test on the Arcane project. Our scope included a detailed security assessment, where we rigorously examined the code to identify vulnerabilities and assess overall efficiency. The testing process involved a meticulous manual review, supplemented by advanced software-assisted techniques, to ensure thorough analysis and accuracy.

Description	**Arcane Project code base**
Platform	**Solana**
Penetration Test Date	**July, 2026**
Repository/Website URL/IP Tested	https://github.com/ArcanePrivacy/Front-end
GitHub Commit Hash (If applicable)	f2c080bf37b49f3b8c404b91df6db350a5ce2717
Fix Review GitHub Commit Hash (If applicable)	3bd29138a8868deaf7c958327d2f1253205a03cf

Note: *Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash", Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.*

Security Rating

After Hashlock's Penetration Test, we found the code base to be **"Secure"**. The code follows simple logic, with correct and detailed ordering.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Penetration Test Findings section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

2 Low-severity vulnerabilities

2 QA

Caution: *Hashlock's Penetration Tests do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system’s performance or security.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Penetration Test Findings

Low

[L-01][arcane-frontend] vite.config.js - The production Content-Security-Policy is weak and is only emitted by the development server

Description

The `Content-Security-Policy` is emitted only by the `Vite` development and preview servers and keeps `'unsafe-eval'`, so a static production build can serve no policy at all on a page that holds raw private keys.

Vulnerability Details

The `script-src` directive keeps `'unsafe-eval'`, and the whole `Content-Security-Policy` is set only through the `Vite` `server.headers` and `preview.headers` options. A static production build served from a CDN emits no policy, and `index.html` carries no fallback. This configuration predates the diff, but the new build adds the raw-key `Arcane Pay` page that runs under it.

Impact

A production deployment may serve no `Content-Security-Policy` at all, and where the policy does apply `'unsafe-eval'` re-enables the `eval` family. On a page that holds raw private keys in memory this weakens the barrier that would otherwise contain an injected script.

Recommendation

Emit a strict `Content-Security-Policy` from the production host or an `index.html` meta fallback, remove `'unsafe-eval'` in favour of a `'wasm-unsafe-eval'`-only policy if the proving stack allows it, and verify the policy is present on the deployed origin.

Status

Acknowledged

[L-02][arcane-frontend] index.html - An integrity-pinned proving script was replaced by an unpinned package, leaving dead loader code behind

Description

An integrity-pinned `snarkjs` CDN script was replaced by the unpinned `arc-sdk-test` package, and the old subresource-integrity loader remains as dead code in `src/lib/zkProof.js` with no caller.

Vulnerability Details

The `index.html` previously loaded `snarkjs` from a CDN with a subresource-integrity hash and `crossorigin`. The new build removes that script and the `snarkjs` dependency, moving proof generation into the unpinned `arc-sdk-test`. The old subresource-integrity loader and `snarkjs` references remain as dead code in `src/lib/zkProof.js` with no caller.

Impact

An integrity-verified, version-locked proving artefact was traded for an unpinned package, and the residual loader code is misleading during maintenance. The substantive supply-chain risk is already captured under the high-severity dependency finding, so what remains here is code hygiene.

Recommendation

Remove the dead subresource-integrity loader and `snarkjs` references from `src/lib/zkProof.js`, and restore an integrity guarantee for the proving path by pinning the replacement dependency.

Status

Resolved

QA

[Q-01][arcane-frontend] src/components/pages/ArcanePayPage.jsx#fetchFeesSimulation - The fee-simulation request has no timeout, error handling, or response validation and re-fires on every keystroke

Description

The fee-simulation request runs with no timeout, error handling, or response validation and re-fires on every keystroke, so a slow or malformed indexer response leaves requests pending or throws during render.

Vulnerability Details

The `fetchFeesSimulation` function calls the indexer `/review` endpoint with no `AbortController`, no timeout, and no `catch`, and the `useEffect` that drives it depends on `totalsByToken`, so it re-runs on every amount edit. Downstream code reads `feeSimulationByIndexer.totalFee.usd` without guarding against a partial response.

Impact

A slow indexer leaves the request pending indefinitely and can trigger unhandled promise rejections, and a malformed response throws while rendering. The keystroke re-fetch is a mild self-inflicted load against the indexer. The effect is confined to the client.

Recommendation

Add an `AbortController` with a timeout, debounce the effect, wrap the call in error handling, and validate the response shape before reading nested fields.

Status

Resolved

[Q-02] [arcane-frontend] .env.example - Bitcoin, Ethereum, and USDC map to the same mint in the example configuration

Description

The `.env.example` maps Bitcoin, Ethereum, and USDC to the same mint value, so the reverse lookup collides the three assets onto one symbol.

Vulnerability Details

The `.env.example` sets `VITE_BITCOIN_MINT`, `VITE_ETHEREUM_MINT`, and `VITE_USDC_MINT` to the same mint value. The reverse lookup `tokenSymbolForMint` resolves a mint to the first matching symbol in `TOKEN_OPTIONS`, so three assets that share a mint collide onto one entry.

Impact

With this example configuration three distinct assets are indistinguishable in labels and receipts. The collision lives in the example placeholders rather than shipped configuration, so the effect is limited to setups that copy the example verbatim.

Recommendation

Use distinct mint values per asset in `.env.example`, and add a startup check that rejects duplicate mints in the token map.

Status

Resolved

Conclusion

After Hashlock's analysis, the Arcane project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our Penetration Tests a collaborative effort. The objective of our security Penetration Tests is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security Penetration Test process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future Penetration Tests. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the code base under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other Penetration Test results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed the code bases in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the code base source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the Penetration Test makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this code base.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Code is deployed and executed on a platform. The platform, its programming language, and other software related to the code base can have their own vulnerabilities that can lead to attacks. Thus, the Penetration Test can't guarantee the explicit security of the Penetration Tested code bases.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au